```
    member = FrequentFlyer.withFrequentFlyerNumber("12345678")
                          .named(firstName,lastName);
    member.setStatus(status);
}
```

❶ Create a new member.

❷ Update the member's status.

Here you reuse the builder method that was introduced in section 10.2.7 to create a new Frequent Flyer member ❶, and then update the status for this member ❷. In doing so, you discover that you need a setter method for the `status` field, which you can simply add to the `FrequentFlyer` class. A simple setter method like this would not justify a separate unit test, as the automated acceptance test is enough to illustrate how this code works.

The next step definition introduces a new concept: you need to keep track of a member's status points:

```
And he has <initialStatusPoints> status points
```

Status points are accumulated for each flight, but the business rules around how many status points are earned for a given flight are complex and vary depending on many factors (the length of the flight, the cabin category, special deals, and so forth). However, a key principle of BDD, and agile development in general, is to keep things simple until you have a good reason to make them more complicated. In this case, the simplest solution for the `FrequentFlyer` class would be to store the current number of status points. Using this approach, the step definition might look like this:

```
@Given("^(?:s?)he has (.*) status points$")
public void earned_status_points(int statusPoints) {
    member.setStatusPoints(statusPoints);
}
```

Again, this is a simple method that doesn't justify any dedicated unit tests.

The next step involves updating the status points of a member account:

```
When he earns <extraPoints> status points
```

Once again, writing the step definition gives you the opportunity to describe the ideal API you'd like to use to perform this operation. For example, you could use a fluent API approach like the one here:

```
@When("^(?:s?)he earns (.*) extra status points$")
public void earn_extra_status_points(int points) {
    member.earns(points).statusPoints();
}
```

This is a more sophisticated API, so you'd typically specify its behavior in more detail in a low-level executable specification, like this one:

```
@Test
public void a_member_should_be_able_to_earn_extra_status_points() {
    FrequentFlyer member =
        = FrequentFlyer.withFrequentFlyerNumber("12345678")
                                        .named("Joe", "Jones");
    member.setStatusPoints(100);
```

Given Joe Jones is a member with 100 status points...